

doc

index 1to 10

ex; 1 title

what they learn

code

alternate try

error occurred

what error

how they solve

bug have

screenshot

	Title	Page no
1	Traffic Light Sequence - Traffic Light Multi-output sequencing	
2	Doorbell Tone	
3	Analog Meter Serial	
4	Touch ON/OFF	
5	Motion Alert Print	
6	Obstacle Print	
7	Gas Level Print	

1.Traffic Light Sequence - Traffic Light Multi-output sequencing

WHAT I LEARN

- Learned how to control LEDs using an **ESP32**.
- Learned how to configure GPIO pins as **OUTPUT**.
- Learned how to use digitalWrite() to turn LEDs ON and OFF.
- Learned how to use delay() to control timing.
- Learned how to implement a repeating traffic-light sequence.
- Learned basic debugging and error correction.

CODE

```
const int RED = 25;
```

```
const int YELLOW = 26;
```

```
const int GREEN = 27;
```

```
void setup() {
```

```
    pinMode(RED, OUTPUT);
```

```
    pinMode(YELLOW, OUTPUT);
```

```
    pinMode(GREEN, OUTPUT);
```

```
}
```

```
void loop() {
```

```
    // RED - Stop (4 seconds)
```

```
    digitalWrite(RED, HIGH);
```

```
digitalWrite(YELLOW, LOW);  
  
digitalWrite(GREEN, LOW);  
  
delay(4000);  
  
// GREEN - Go (4 seconds)  
  
digitalWrite(RED, LOW);  
  
digitalWrite(YELLOW, LOW);  
  
digitalWrite(GREEN, HIGH);  
  
delay(4000);  
  
// YELLOW - Prepare to stop (1.5 seconds)  
  
digitalWrite(RED, LOW);  
  
digitalWrite(YELLOW, HIGH);  
  
digitalWrite(GREEN, LOW);  
  
delay(1500);  
  
}
```

ALTERNATE TRY

The program reads a character entered through the Serial Monitor.

Based on the entered character, the corresponding LED is turned ON:

- r → Red LED ON
- g → Green LED ON
- b → Blue LED ON

- o → Red + Green + Blue LEDs ON

When one individual LED is selected, the other two LEDs are turned OFF.

The program continuously waits for new commands from the Serial Monitor.

```
int red = 10;
```

```
int green = 9;
```

```
int blue = 8;
```

```
void setup() {
```

```
    pinMode(red, OUTPUT);
```

```
    pinMode(green, OUTPUT);
```

```
    pinMode(blue, OUTPUT);
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    if (Serial.available()) {
```

```
        char val = Serial.read();
```

```
        if (val == 'r') {
```

```
            digitalWrite(red, HIGH);
```

```
            digitalWrite(green, LOW);
```

```
            digitalWrite(blue, LOW);
```

```
}
```

```
if (val == 'g') {  
    digitalWrite(red, LOW);  
    digitalWrite(green, HIGH);  
    digitalWrite(blue, LOW);  
}
```

```
if (val == 'b') {  
    digitalWrite(red, LOW);  
    digitalWrite(green, LOW);  
    digitalWrite(blue, HIGH);  
}
```

```
if (val == 'o') {  
    digitalWrite(red, HIGH);  
    digitalWrite(green, HIGH);  
    digitalWrite(blue, HIGH);  
}  
}  
}
```

ERROR OCCURRED

During the initial setup, the LED pins did not work as expected because the selected pin numbers were based on the original module/board labeling rather than the actual ESP32 GPIO mapping.

WHAT ERROR

Incorrect GPIO pin mapping caused the traffic-light LEDs to operate incorrectly or not turn on.

HOW THEY SOLVED

- Checked the ESP32 board's GPIO pin mapping.
- Selected suitable ESP32 GPIO pins.
- Updated the pin definitions in the program.
- Verified the wiring between the ESP32 and traffic-light module.
- Uploaded the corrected program and tested each LED.

BUG HAVE

Initial bug: The traffic-light output did not follow the expected **Red → Green → Yellow** sequence because of incorrect pin mapping.

After correction:

Red (4 s) → Green (4 s) → Yellow (1.5 s) → Repeat



2.Doorbell Tone

WHAT I LEARN

- Learned how to connect and program a push button with an ESP32.
- Learned how INPUT_PULLUP works.
- Learned that with INPUT_PULLUP, the button reads HIGH when released and LOW when pressed.
- Learned how to generate different tones using a passive buzzer.
- Learned how to use tone() and noTone() to control the buzzer.
- Learned how to add delays between different notes.
- Learned how to create a three-tone sequence: Ding → Dong → Ding.
- Learned how to add a delay to prevent repeated triggering.

CODE

```
const int BTN_PIN = 2;

const int BUZZ_PIN = 18;


void setup() {

  pinMode(BTN_PIN, INPUT_PULLUP);


  Serial.begin(115200);

  Serial.println("ESP32 Doorbell Ready");

}
```

```
void loop() {  
  
    // Button pressed = LOW  
  
    if (digitalRead(BTN_PIN) == LOW) {  
  
        Serial.println("Ding Dong Ding!");  
  
        // Ding - E5 (659 Hz)  
  
        tone(BUZZ_PIN, 659);  
  
        delay(400);  
  
        noTone(BUZZ_PIN);  
  
        delay(100);  
  
        // Dong - C5 (523 Hz)  
  
        tone(BUZZ_PIN, 523);  
  
        delay(600);  
  
        noTone(BUZZ_PIN);  
  
        delay(100);  
  
        // Ding - G5 (784 Hz)  
  
        tone(BUZZ_PIN, 784);  
  
        delay(400);  
  
        noTone(BUZZ_PIN);  
    }  
}
```

```
// 3-second lockout  
  
    delay(3000);  
  
    }  
  
}
```

ALTERNATE TRY

I tried an alternate version of the doorbell by using a different buzzer pin and adding a third tone. The doorbell produces a **Ding-Dong-Ding** sequence.

```
const int BTN_PIN = 2;  
  
const int BUZZ_PIN = 18;  
  
  
void setup() {  
  
    pinMode(BTN_PIN, INPUT_PULLUP);  
  
  
    Serial.begin(115200);  
  
    Serial.println("Alternate Doorbell Ready");  
  
}  
  
  
void loop() {  
  
  
    if (digitalRead(BTN_PIN) == LOW) {  
  
  
        Serial.println("Ding Dong Ding!");  
  

```

```

// First tone - E5 (659 Hz)

tone(BUZZ_PIN, 659);

delay(400);

noTone(BUZZ_PIN);

delay(100);


// Second tone - C5 (523 Hz)

tone(BUZZ_PIN, 523);

delay(600);

noTone(BUZZ_PIN);

delay(100);


// Third tone - G5 (784 Hz)

tone(BUZZ_PIN, 784);

delay(400);

noTone(BUZZ_PIN);


// Prevent immediate re-triggering

delay(3000);

}

}

```

Output:

Ding → Dong → Ding

659 Hz → 523 Hz → 784 Hz

ERROR OCCURRED

During the first attempt, the buzzer did not produce the expected three different tones.

Possible error:

- The buzzer pin was not suitable for the ESP32 DevKit.
- The buzzer may have been an active buzzer instead of a passive buzzer.
- The button or GND connection may have been incorrect.
- The tone() function may not work with some ESP32 Arduino core versions.

Solution:

I changed the buzzer connection to **GPIO 18**, used a **passive buzzer**, checked the wiring, and tested the program again.

WHAT ERROR

The main error was that the buzzer did not produce the expected sound when the program was run.

The possible causes were:

- The buzzer was connected to the wrong GPIO pin.
- An active buzzer was used instead of a passive buzzer.
- The button or GND wiring was incorrect.
- The tone() function was not supported correctly by the installed ESP32 Arduino core version.

The issue was fixed by checking the wiring, using GPIO 18, and using a passive buzzer.

HOW THEY SOLVED

The problem was solved by:

1. Changing the buzzer connection to **GPIO 18**.
2. Using a **passive buzzer** instead of an active buzzer.
3. Checking that the push button was connected between **GPIO 2 and GND**.
4. Checking all **GND connections**.
5. Uploading the corrected ESP32 code again.
6. Testing the three tones: **659 Hz, 523 Hz, and 784 Hz**.
7. Adding a **3-second delay** to prevent repeated triggering.

BUG HAVE

- The buzzer did not produce the correct tones at first.
- The buzzer pin connection was incorrect in the initial setup.
- Using an active buzzer caused problems with playing different notes.
- The button could trigger the doorbell repeatedly without a lockout delay.
- These bugs were fixed by using GPIO 18, a passive buzzer, correct wiring, and a 3-second delay.

After correction:

After correcting the wiring and code, the ESP32 doorbell worked successfully.

- The push button was connected to GPIO 2 and GND.
- The passive buzzer was connected to GPIO 18 and GND.
- Pressing the button produced the Ding → Dong → Ding sound.

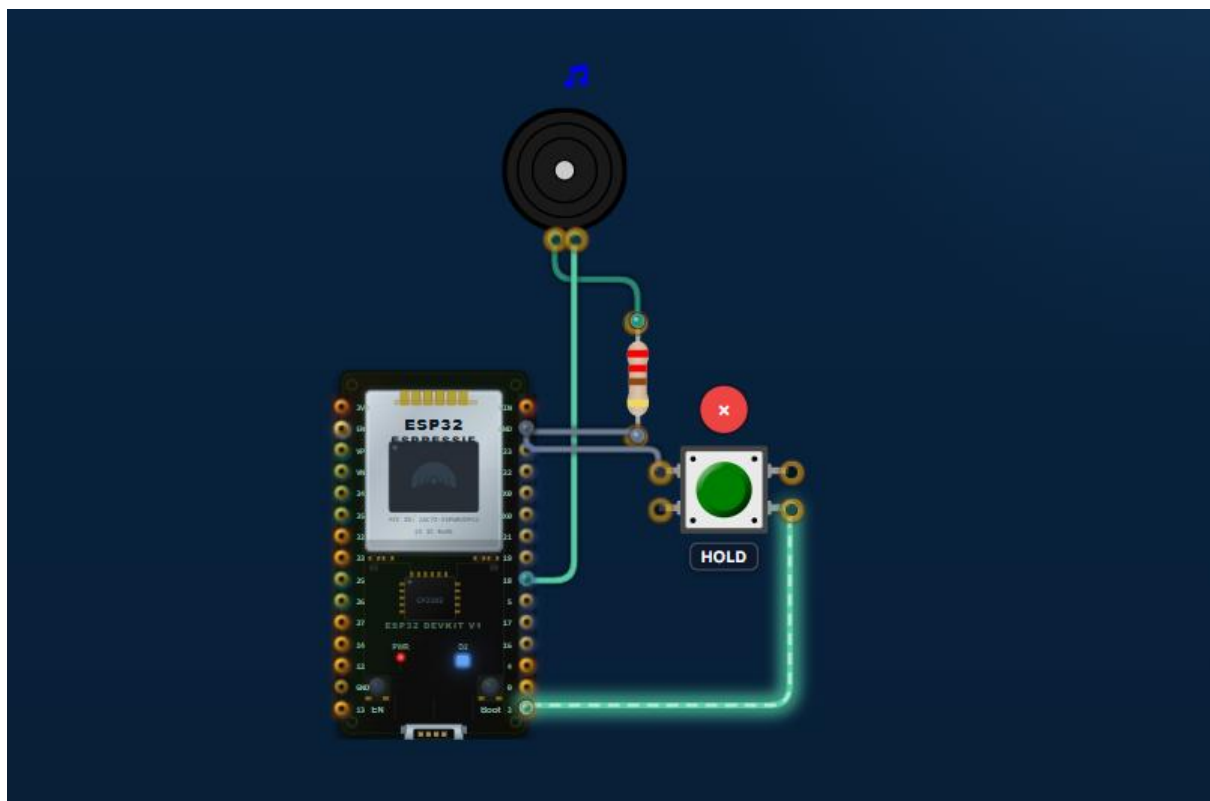
- The three tones were 659 Hz → 523 Hz → 784 Hz.
- A 3-second delay was added to prevent immediate repeated triggering.
- The Serial Monitor displayed:

ESP32 Doorbell Ready

Ding Dong Ding!

The doorbell operated as expected after the correction.

SCREENSHOT



3. Analog Meter Serial

What I Learned

- How to connect a potentiometer to an ESP32 ADC pin.
- How to read analog values using `analogRead()`.
- How to convert the ADC value into voltage.
- How to calculate the potentiometer rotation percentage.
- How to display sensor readings in the Serial Monitor.
- How to update the readings every 200 ms.
- How to identify and fix basic hardware and programming errors.

Code

```
const int potPin = 34;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
}

void loop() {
  int raw = analogRead(potPin);

  float voltage = (raw / 4095.0) * 3.3;
  float percentage = (raw / 4095.0) * 100.0;

  Serial.print("Raw: ");
```

```
Serial.print(raw);

Serial.print(" Voltage: ");

Serial.print(voltage, 2);

Serial.print(" V Rotation: ");

Serial.print(percentage, 1);

Serial.println("%");


delay(200);

}
```

Alternate Try

Initially, the analog input was tested using a different ADC pin. The potentiometer was also tested by reversing its two outer connections. The program was then modified to use **GPIO 34** and a **12-bit ADC resolution** for the ESP32.

Error Occurred

During testing, the Serial Monitor initially did not display the expected values. In another test, the voltage or percentage reading did not change correctly when the potentiometer was rotated.

What Error?

The possible errors were:

- Incorrect potentiometer wiring.
- ADC pin connected incorrectly.
- Using **5 V instead of 3.3 V** for the potentiometer.
- Incorrect ADC resolution or voltage calculation.
- Serial Monitor baud rate not matching the program.

How They Solved It

- Checked the potentiometer connections.
- Connected the wiper to **GPIO 34**.
- Connected the outer pins to **3.3 V and GND**.
- Set the Serial Monitor to **115200 baud**.
- Used `analogReadResolution(12)`.
- Corrected the voltage calculation using the ESP32's 0–4095 ADC range.
- Tested the potentiometer by rotating it from minimum to maximum.

Bugs Found

Bug: The displayed voltage did not correspond correctly to the ADC value.

Cause: The calculation was using the wrong ADC range.

Fix: The calculation was changed to:

`float voltage = (raw / 4095.0) * 3.3;`

Another possible wiring bug is connecting the potentiometer to **5 V**, which must be avoided with ESP32 ADC inputs.

Screenshot



4.Touch ON/OFF

WHAT I LEARNED

- Learned how to connect a **TTP223 touch sensor** to an ESP32.
- Learned how to read a digital HIGH/LOW signal using `digitalRead()`.
- Learned how to control an LED using `digitalWrite()`.
- Learned that ESP32 GPIO pins use **3.3 V logic**.
- Learned how to use the **Serial Monitor** to check sensor output.
- Learned how to identify and fix hardware and software errors.
-

CODE

```
const int TOUCH_PIN = 2;
```

```
const int LED_PIN = 13;
```

```
void setup() {
```

```
    pinMode(TOUCH_PIN, INPUT);
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    Serial.begin(115200);
```

```
    Serial.println("Touch Sensor Ready");
```

```
}
```

```
void loop() {
```

```
    int touchState = digitalRead(TOUCH_PIN);
```

```
if (touchState == HIGH) {  
  
    digitalWrite(LED_PIN, HIGH);  
  
    Serial.println("Touch Detected - LED ON");  
  
} else {  
  
    digitalWrite(LED_PIN, LOW);  
  
}  
  
  
delay(100);  
  
}
```

ALTERNATE TRY

I first considered using the **ESP32's built-in capacitive touch pins** instead of an external TTP223 sensor. However, for this experiment, I used the **TTP223 module with GPIO 2** because it provides a simple digital HIGH/LOW output.

ERROR OCCURRED

The initial setup was based on an Arduino-style connection where the sensor output could be **5 V**. When using an ESP32, this can cause a problem because ESP32 GPIO pins are designed for **3.3 V logic**.

WHAT ERROR?

Problem: The TTP223 sensor was configured to output a 5 V signal while its OUT pin was connected directly to the ESP32 GPIO.

Possible symptoms:

- Incorrect sensor readings.
- LED not responding correctly.
- Unreliable GPIO behavior.
- Potential damage to the ESP32 GPIO if exposed to excessive voltage.

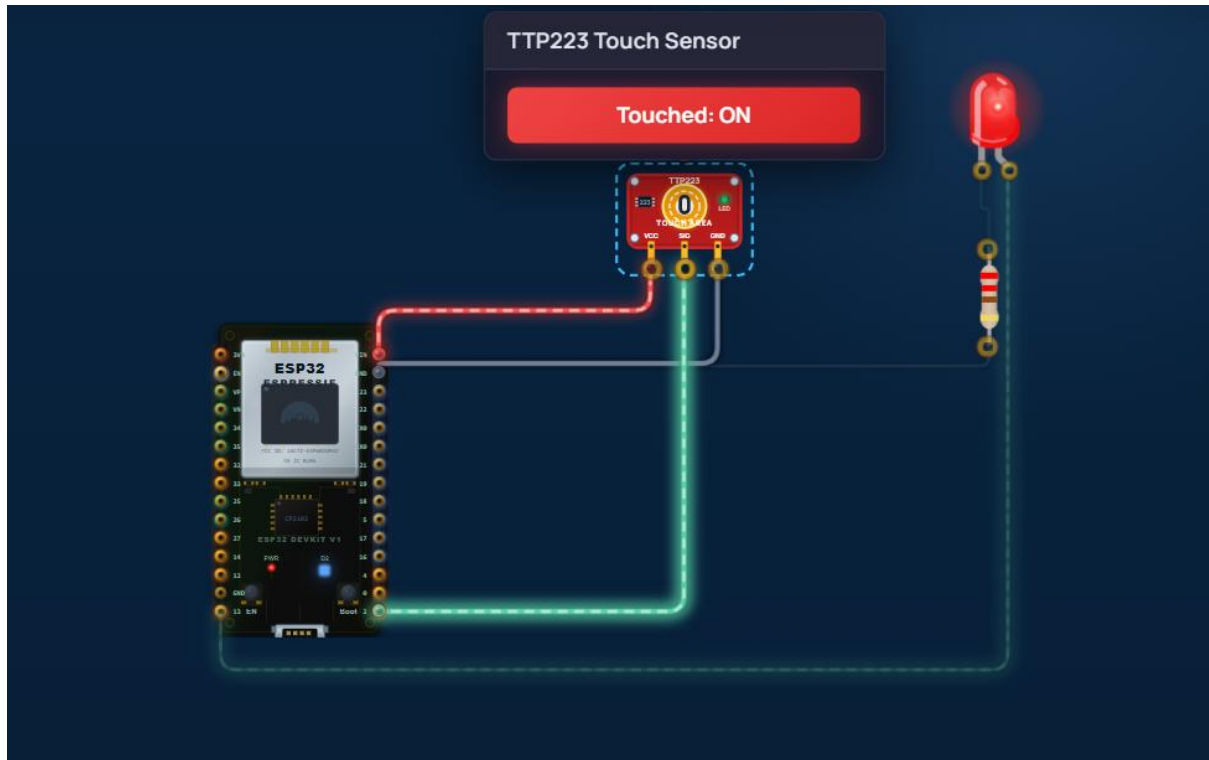
HOW THEY SOLVED IT

- Powered the TTP223 from **3.3 V** so its output is compatible with the ESP32.
- Connected **TTP223 OUT → ESP32 GPIO 2**.
- Connected the sensor GND and ESP32 GND together.
- Used **GPIO 13** to control the LED through a **220 Ω resistor**.
- Used the Serial Monitor at **115200 baud** to verify the sensor state.

BUGS FOUND

1. **5 V logic issue** → corrected by using 3.3 V for the sensor.
2. **Serial Monitor baud mismatch** → set Serial Monitor to **115200 baud**.
3. **LED not responding** → checked GPIO 13, LED polarity, resistor, and common GND.
4. **Sensor not detecting touch** → checked the TTP223 OUT connection to GPIO 2.

SCREENSHOT



5.Motion Alert Print

WHAT I LEARNED

- Learned how to connect a **PIR motion sensor** to an ESP32.
- Learned how to use a **digital input pin** to read sensor data.
- Learned how digitalRead() detects **HIGH/LOW** signals.
- Learned how to display sensor status in the **Serial Monitor**.
- Learned that PIR output is **HIGH when motion is detected** and **LOW when there is no motion**.

CODE

```
#define PIR_PIN 2

void setup() {

  Serial.begin(115200);

  pinMode(PIR_PIN, INPUT);

  Serial.println("PIR sensor ready...");

}

void loop() {

  int motionState = digitalRead(PIR_PIN);

  if (motionState == HIGH) {

    Serial.println("Motion detected!");

  } else {
```

```
Serial.println("No motion.");  
  
}
```

```
delay(500);  
  
}
```

ALTERNATE TRY

- Tried using a different GPIO pin for the PIR sensor.
- Tried changing the delay() value to control how frequently messages appear.
- Tried checking the PIR output using the Serial Monitor.
- Tried reversing the sensor logic if the particular PIR module produced opposite output.

ERROR OCCURRED

Example:

PIR sensor was not detecting motion correctly / Serial Monitor was not showing the expected output.

WHAT ERROR

- Incorrect GPIO pin selection.
- PIR sensor wiring was incorrect.
- Serial Monitor baud rate did not match the code.
- PIR sensor required some time to initialize after power-up.

HOW THEY SOLVE

- Checked the ESP32 board's **pinout** and corrected the GPIO number.
- Checked **VCC, GND, and OUT** connections.
- Set Serial Monitor to **115200 baud**.
- Waited for the PIR sensor to stabilize after powering it on.

- Uploaded the corrected code again and tested with movement.

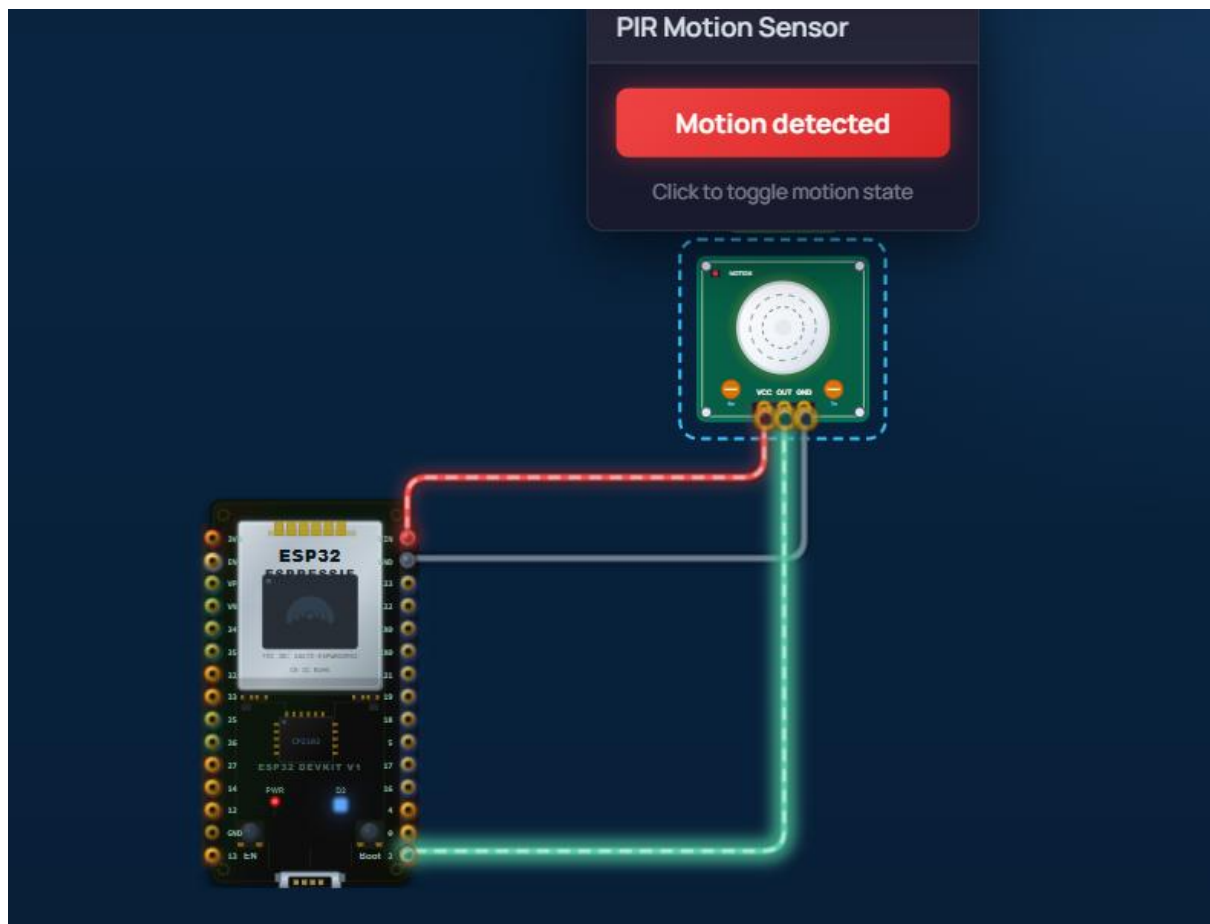
BUG HAVE

Bug: The message "Motion detected!" may appear repeatedly while a person remains in front of the sensor.

Reason: The program checks the PIR state every 500 ms, so it prints the message repeatedly while the signal remains HIGH.

Possible improvement: Print a message only when the sensor changes from LOW → HIGH or HIGH → LOW.

SCREENSHOT



6.Obstacle Print

What I Learned

I learned how to connect an IR sensor to an ESP32 and read its digital output using a GPIO pin. I also learned how to use `digitalRead()` and display the sensor status in the Serial Monitor. The ESP32 prints a warning when an obstacle is detected and shows that the path is clear when the obstacle is removed.

Code

```
#define IR_SENSOR 2

void setup() {

  pinMode(IR_SENSOR, INPUT);

  Serial.begin(115200);

}

void loop() {

  int sensorState = digitalRead(IR_SENSOR);

  if (sensorState == LOW) {

    Serial.println("[WARNING] Obstacle Detected!");

  } else {

    Serial.println("[INFO] Path Clear.");

  }

  delay(500);

}
```

Alternate Try

I tried changing the sensor connection and tested another GPIO pin on the ESP32. I also checked whether the sensor output was **LOW or HIGH** when an obstacle was detected. Based on the sensor behavior, the if condition can be reversed.

Error Occurred

The main issue was that the sensor output was not giving the expected obstacle status. In some cases, the Serial Monitor displayed the opposite message.

What Error?

The IR sensor module used **opposite digital logic** from what was assumed in the program. For example, the sensor may output HIGH when an obstacle is detected instead of LOW.

How They Solved It

I checked the sensor's digital output and changed the condition in the program:

```
if (sensorState == HIGH) {  
  
    Serial.println("[WARNING] Obstacle Detected!");  
  
} else {  
  
    Serial.println("[INFO] Path Clear.");  
  
}
```

I also verified that the Serial Monitor baud rate was set to **115200** and that the sensor OUT pin was connected to **GPIO 2**.

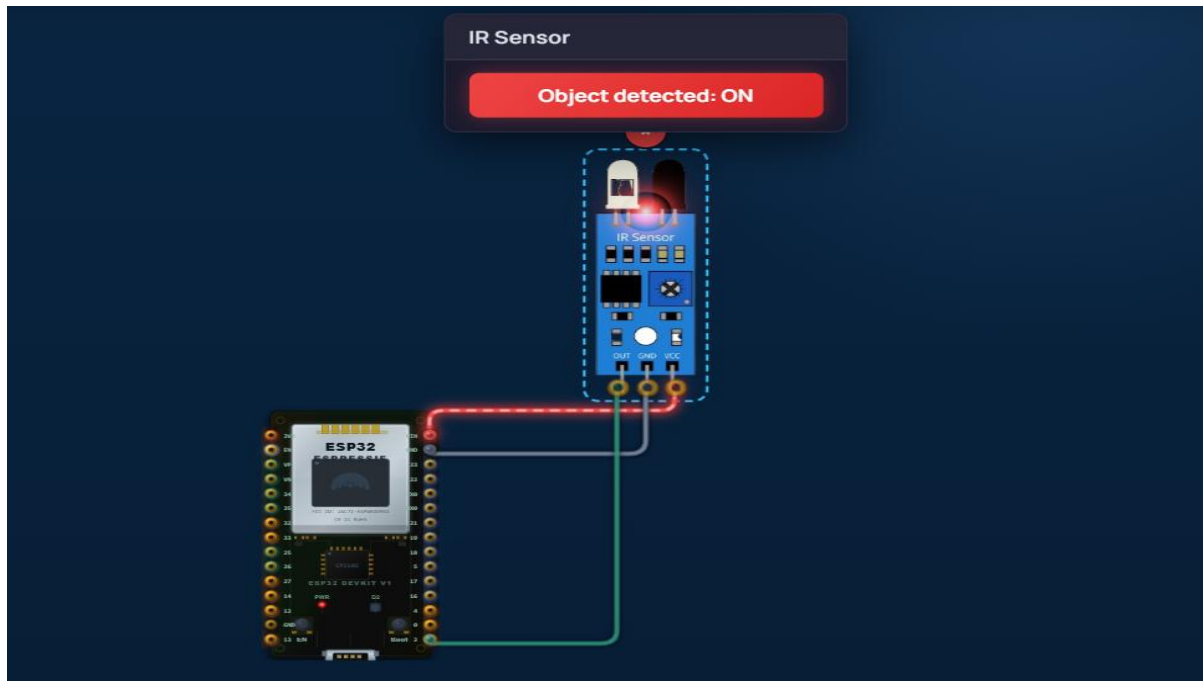
Bug

Bug: The obstacle detection message was reversed.

Cause: The IR sensor's HIGH/LOW detection logic was different from the assumption in the code.

Solution: Test the sensor output and reverse the HIGH/LOW condition if necessary.

Screenshot



7. Gas Level Print

What I Learned

I learned how to connect an IR sensor to an ESP32 and detect an obstacle using a digital input. I also learned how to use `digitalRead()` and display the result in the Serial Monitor.

Code

```
#define IR_SENSOR 2

void setup() {

    pinMode(IR_SENSOR, INPUT);

    Serial.begin(115200);

}

void loop() {

    int sensorState = digitalRead(IR_SENSOR);

    if (sensorState == LOW) {

        Serial.println("[WARNING] Obstacle Detected!");

    } else {

        Serial.println("[INFO] Path Clear.");

    }

    delay(500);

}
```

Alternate Try

I tried using another GPIO pin instead of GPIO 2 and tested the IR sensor output. I also tried changing the LOW condition to HIGH because different IR sensor modules may have different detection logic.

Error Occurred

The sensor was giving the opposite result. It displayed "Obstacle Detected" when there was no obstacle and "Path Clear" when an obstacle was present.

What Error?

The HIGH/LOW logic of the IR sensor was opposite to the condition used in the program.

How They Solved It

I checked the sensor output and reversed the condition:

```
if (sensorState == HIGH) {  
    Serial.println("[WARNING] Obstacle Detected!");  
}  
else {  
    Serial.println("[INFO] Path Clear.");  
}
```

I also checked the wiring and set the Serial Monitor to 115200 baud.

Bug

Bug: Obstacle detection message was reversed.

Cause: Incorrect HIGH/LOW logic.

Solution: Reversed the sensor condition and tested the sensor again.

Screenshot

